

Name: HARSHITHA A B

RegNo: 727723eucb018

Sub: Professional Elective 3

Customer Segmentation and Market Basket Intelligence Platform

1. Abstract

In the competitive e-commerce industry, understanding customer behavior is essential for improving customer satisfaction and increasing sales. This project focuses on customer segmentation and market basket intelligence using unsupervised machine learning techniques. Clustering algorithms such as K-Means, Hierarchical Clustering, and DBSCAN are applied to identify distinct customer groups and detect anomalies. Principal Component Analysis (PCA) is used for dimensionality reduction and visualization. The insights obtained from this analysis help businesses create personalized marketing campaigns, improve customer retention, and optimize product recommendations.

2. Problem Statement

E-commerce companies often collect large volumes of customer transaction data but struggle to extract meaningful insights from it. Traditional marketing approaches treat all customers similarly, resulting in lower engagement and reduced conversion rates.

The objective of this project is to develop an unsupervised learning system that segments customers based on purchasing behavior and identifies hidden patterns within transaction data.

3. Objectives

- Perform customer segmentation using clustering techniques.
- Detect noise and anomalies in customer behavior.
- Reduce data dimensionality using PCA.
- Visualize customer groups effectively.
- Generate business-friendly customer personas.
- Discover product purchasing patterns through market basket analysis.

4. Dataset Description

Dataset Used: Online Retail Dataset

The dataset contains transaction records from an online retail company. Each record includes customer information, product details, purchase quantity, unit price, and transaction timestamps.

Features Used

- Customer ID
- Invoice Number
- Invoice Date
- Product Description
- Quantity
- Unit Price
- Total Purchase Amount

5. Methodology

Step 1: Data Collection

The Online Retail Dataset was imported into the analysis environment using Python and Pandas.

Step 2: Data Cleaning

The following preprocessing steps were performed:

- Removed missing Customer IDs.
- Removed records with negative quantities.
- Removed records with invalid prices.
- Created a Total Amount feature.

Step 3: Feature Engineering

RFM Analysis was used to represent customer behavior:

Recency

Number of days since the customer's last purchase.

Frequency

Number of transactions made by the customer.

Monetary Value

Total amount spent by the customer.

Step 4: Data Standardization

The RFM features were standardized using StandardScaler to ensure equal contribution during clustering.

Step 5: Dimensionality Reduction

Principal Component Analysis (PCA) was applied to reduce dimensions and improve visualization.

Step 6: Customer Clustering

The following clustering algorithms were implemented:

- K-Means Clustering
- Hierarchical Clustering
- DBSCAN

Step 7: Market Basket Analysis

Association Rule Mining was performed to identify frequently purchased product combinations.

6. Algorithms Used

K-Means Clustering

K-Means partitions customers into distinct clusters by minimizing intra-cluster variance. It is efficient and widely used for customer segmentation.

Hierarchical Clustering

Hierarchical Clustering builds a hierarchy of clusters and helps understand customer relationships through dendrogram visualization.

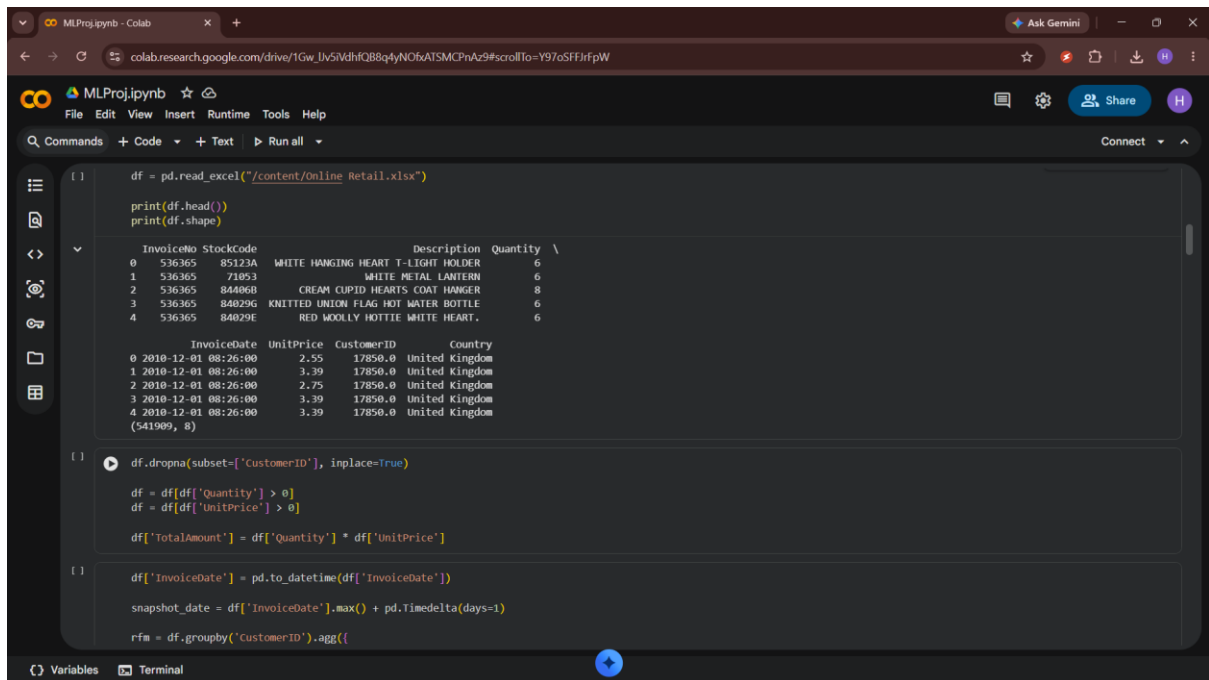
DBSCAN

DBSCAN identifies dense regions of data and labels sparse points as noise or anomalies. This makes it useful for detecting unusual customer behavior.

Principal Component Analysis (PCA)

PCA reduces dimensionality while preserving important information, enabling effective visualization of customer clusters.

7. Results Analysis and Output



The screenshot shows a Google Colab notebook with the following code and output:

```
[1] df = pd.read_excel("/content/Online_Retail.xlsx")

print(df.head())
print(df.shape)
```

	InvoiceNo	StockCode	Description	Quantity	
0	536365	85123A	WHITE HANGING HEART T-LIGHT HOLDER	6	
1	536365	71053	WHITE METAL LANTERN	6	
2	536365	844068	CREAM CUPID HEARTS COAT HANGER	8	
3	536365	84029G	KNITTED UNION FLAG HOT WATER BOTTLE	6	
4	536365	84029E	RED WOOLLY HOTTIE WHITE HEART.	6	

```
[1] df.dropna(subset=['CustomerID'], inplace=True)

df = df[df['Quantity'] > 0]
df = df[df['UnitPrice'] > 0]

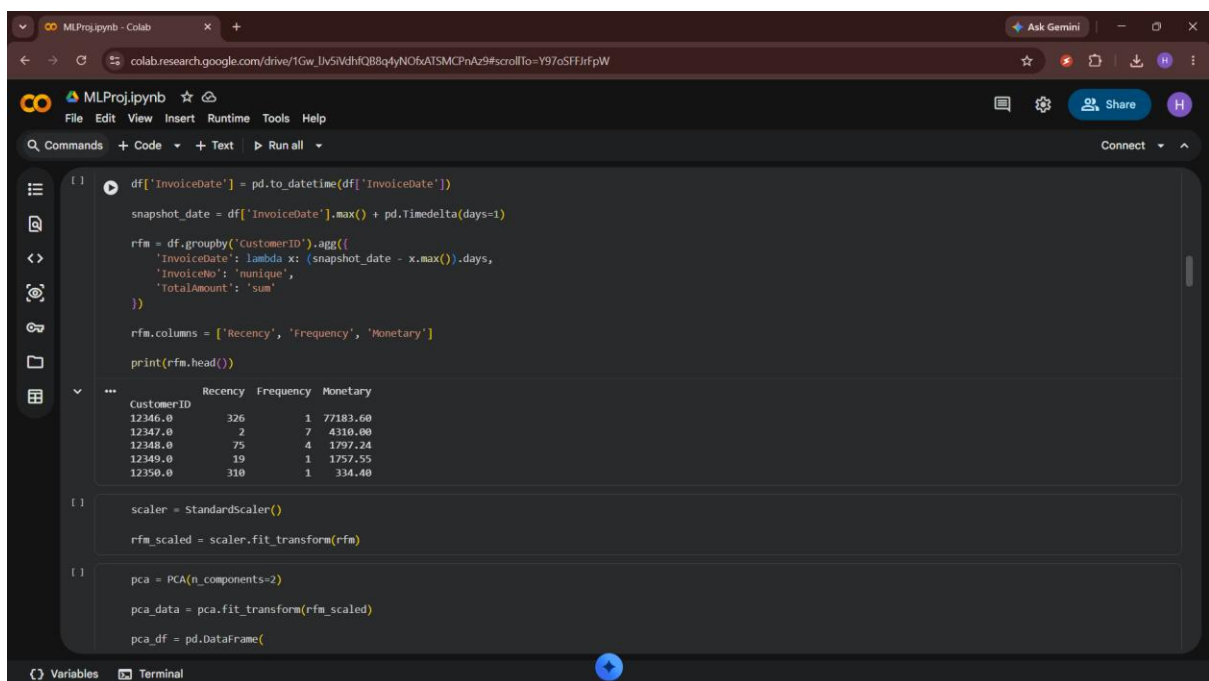
df['TotalAmount'] = df['Quantity'] * df['UnitPrice']

[1] df['InvoiceDate'] = pd.to_datetime(df['InvoiceDate'])

snapshot_date = df['InvoiceDate'].max() + pd.Timedelta(days=1)

rfm = df.groupby('customerID').agg({
```

Variables Terminal



The screenshot shows the continuation of the Google Colab notebook with the following code and output:

```
[1] df['InvoiceDate'] = pd.to_datetime(df['InvoiceDate'])

snapshot_date = df['InvoiceDate'].max() + pd.Timedelta(days=1)

rfm = df.groupby('customerID').agg({
    'InvoiceDate': lambda x: (snapshot_date - x.max()).days,
    'InvoiceNo': 'nunique',
    'TotalAmount': 'sum'
})

rfm.columns = ['Recency', 'Frequency', 'Monetary']

print(rfm.head())
```

	Recency	Frequency	Monetary
CustomerID			
12346.0	326	1	77183.60
12347.0	2	7	4310.00
12348.0	75	4	1797.24
12349.0	19	1	1757.55
12350.0	310	1	334.40

```
[1] scaler = StandardScaler()

rfm_scaled = scaler.fit_transform(rfm)

[1] pca = PCA(n_components=2)

pca_data = pca.fit_transform(rfm_scaled)

pca_df = pd.DataFrame(
```

Variables Terminal

```
MLProj.ipynb - Colab
colab.research.google.com/drive/1Gw_lh5ivdhfQB8q4yNO6xATSMCPnAz9#scrollTo=Y97oSFfJfPw
MLProj.ipynb
File Edit View Insert Runtime Tools Help
Commands + Code + Text Run all
Connect

1
pca = PCA(n_components=2)
pca_data = pca.fit_transform(rfm_scaled)

pca_df = pd.DataFrame(
    pca_data,
    columns=['PC1', 'PC2']
)

print(pca.explained_variance_ratio_)
[0.55502102 0.30249315]

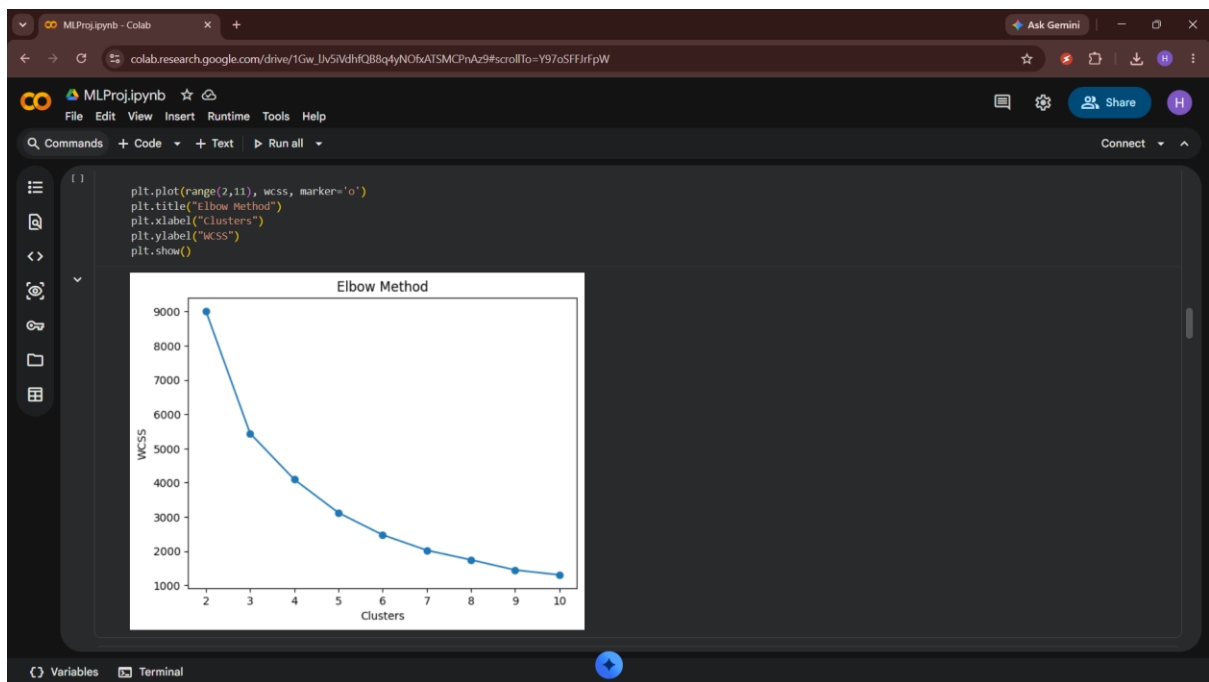
2
wcss = []

for i in range(2,11):
    kmeans = KMeans(
        n_clusters=i,
        random_state=42,
        n_init=10
    )

    kmeans.fit(rfm_scaled)

    wcss.append(kmeans.inertia_)

plt.plot(range(2,11), wcss, marker='o')
plt.title("Elbow Method")
plt.xlabel("Clusters")
plt.ylabel("WCSS")
plt.show()
```



```
MLProj.ipynb - Colab
colab.research.google.com/drive/1Gw_Lh5iVdhfQB8q4yNO6xATSMCPnAz9#scrollTo=Y97oSFfIrfpW
Ask Gemini
MLProj.ipynb
File Edit View Insert Runtime Tools Help
Commands + Code + Text Run all
Connect

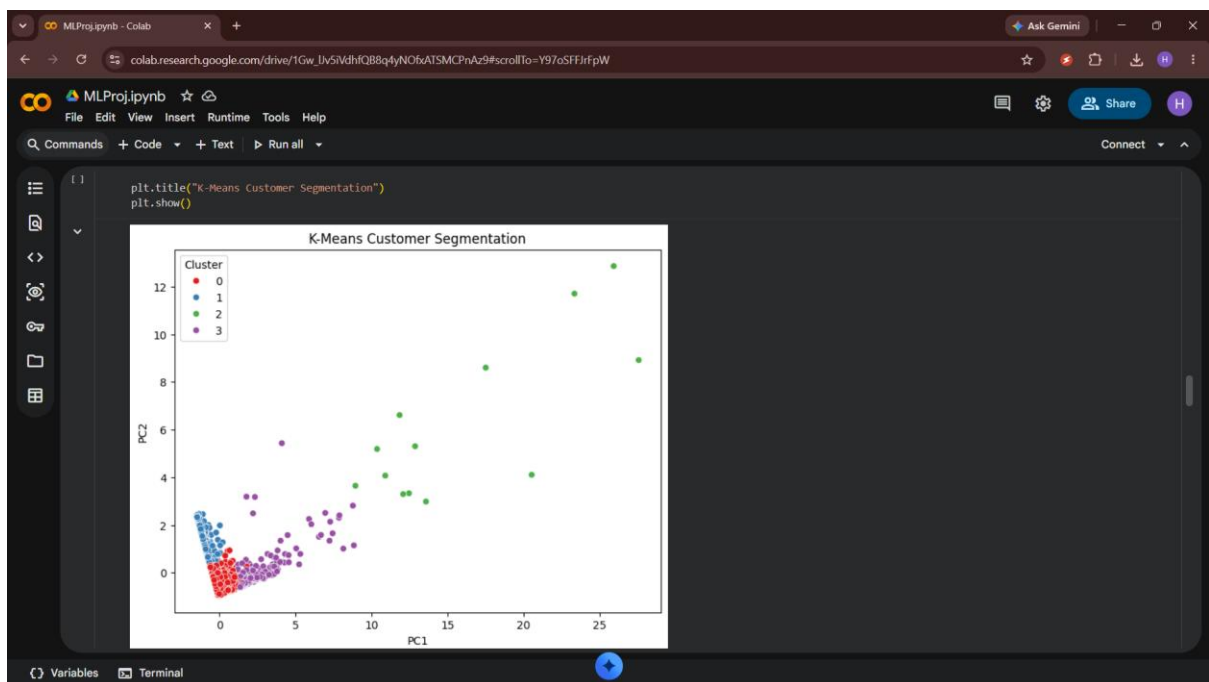
1 kmeans = KMeans(
    n_clusters=4,
    random_state=42,
    n_init=10
)

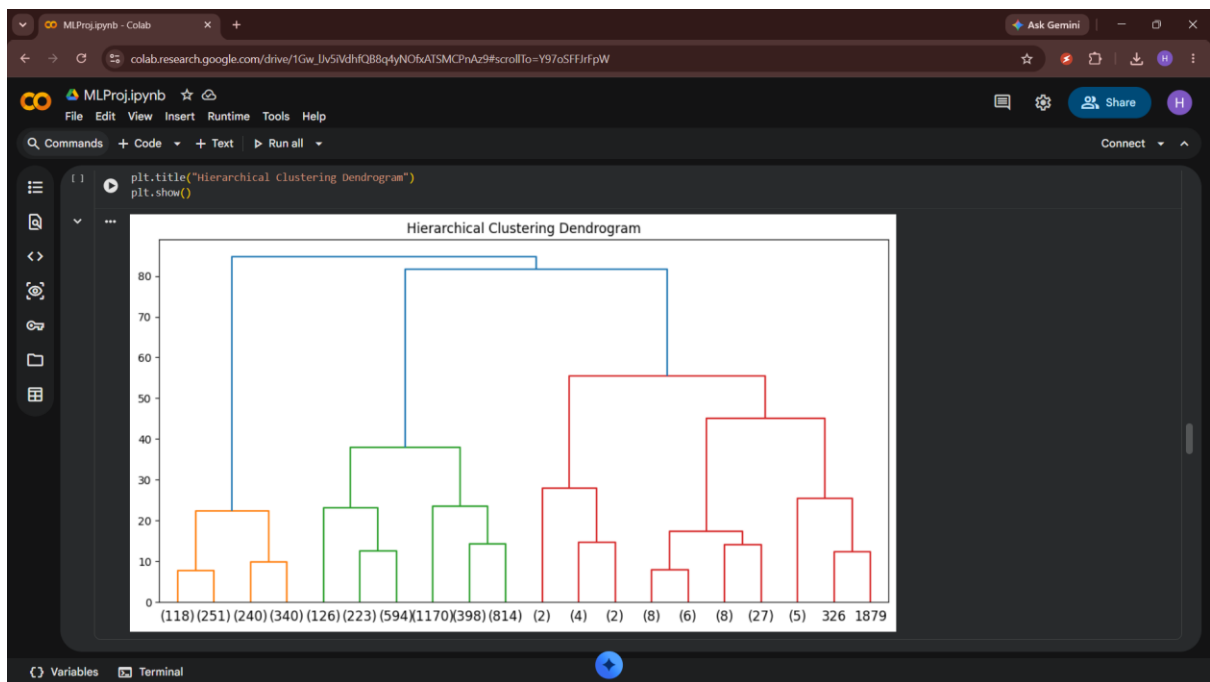
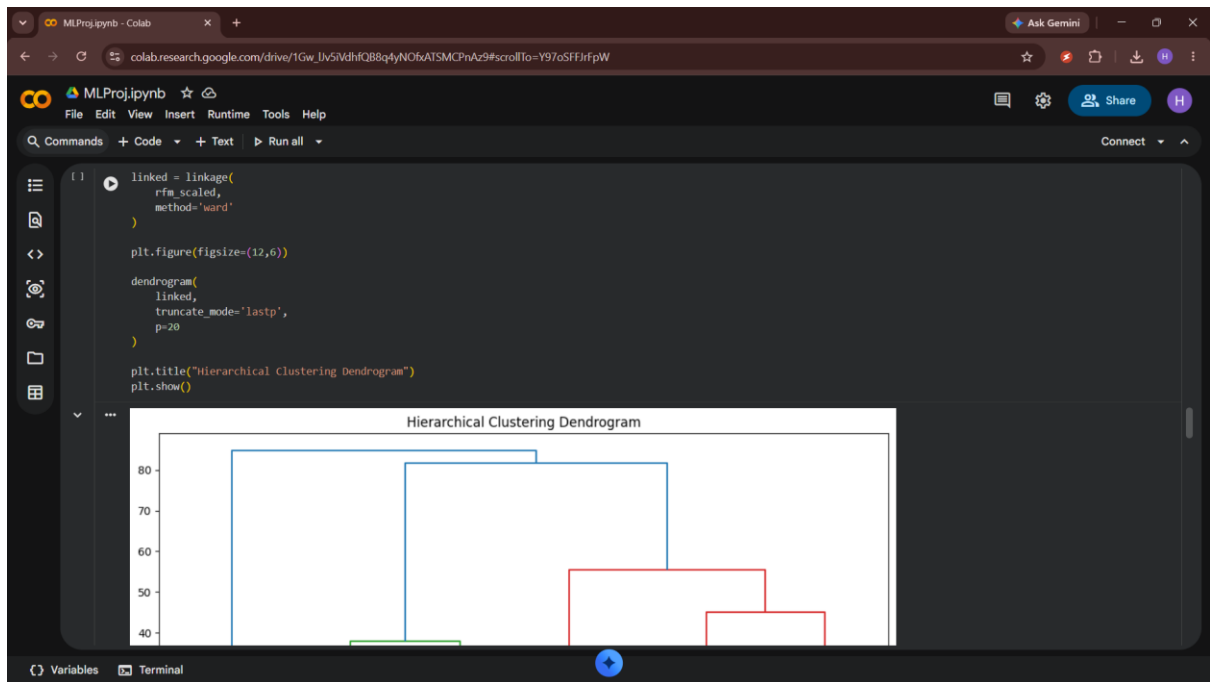
clusters = kmeans.fit_predict(rfm_scaled)
rfm['KMeans_Cluster'] = clusters
print(rfm.head())

CustomerID Recency Frequency Monetary KMeans_Cluster
12346.0 326 1 77183.60 3
12347.0 2 7 4310.00 0
12348.0 75 4 1797.24 0
12349.0 19 1 1757.55 0
12350.0 310 1 334.40 1

2 pca_df['Cluster'] = clusters
plt.figure(figsize=(8,6))

sns.scatterplot(
    data=pca_df,
    x='PC1',
    y='PC2',
    hue='Cluster',
    palette='Set1'
)
```





```
dbscan = DBSCAN(
    eps=0.7,
    min_samples=5
)

db_labels = dbscan.fit_predict(rfm_scaled)

rfm['DBSCAN_Cluster'] = db_labels

print(rfm['DBSCAN_Cluster'].value_counts())

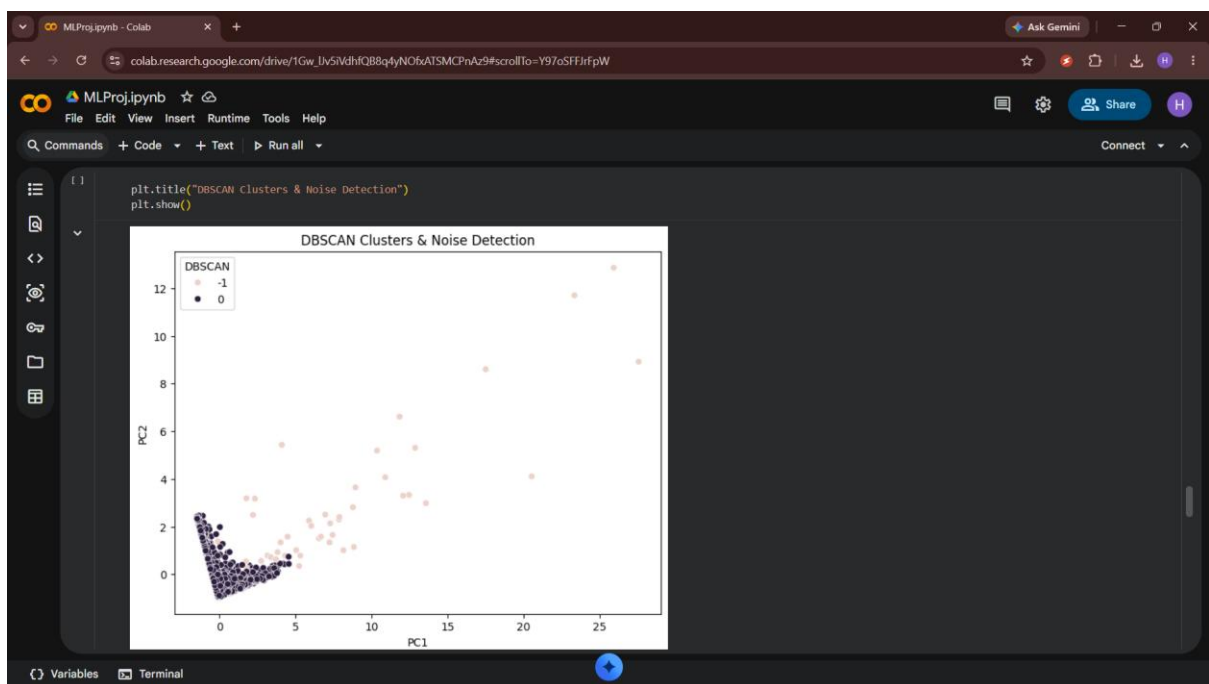
DBSCAN_Cluster
0    4293
-1     45
Name: count, dtype: int64

pca_df['DBSCAN'] = db_labels

plt.figure(figsize=(8,6))

sns.scatterplot(
    data=pca_df,
    x='PC1',
    y='PC2',
    hue='DBSCAN'
)

plt.title("DBSCAN Clusters & Noise Detection")
plt.show()
```




```
MLProj.ipynb - Colab
colab.research.google.com/drive/1Gw_Lh5VdhfQB8q4yNO6xATSMCPnAz9#scrollTo=Y97oSFfIfpW
Ask Gemini
File Edit View Insert Runtime Tools Help
Commands + Code + Text Run all
Connect

1 cluster_report = rfm.groupby(
    "KMeans_Cluster"
).mean()
print(cluster_report)

KMeans_Cluster  Recency  Frequency  Monetary  DBSCAN_Cluster
0              43.702685    3.682711    1359.049284         -0.000327
1             248.075914    1.552015     480.617480         -0.000937
2              7.384615    82.538462    127338.313846         -1.000000
3             15.500000    22.333333    12709.090490         -0.147059

2 basket = (df
    .groupby(['InvoiceNo', 'Description'])['Quantity']
    .sum()
    .unstack()
    .fillna(0))

    basket = basket.applymap(
        lambda x: 1 if x > 0 else 0
    )

3 frequent_items = apriori(
    basket,
    min_support=0.02,
    use_colnames=True
)

print(frequent_items.head())

support  itemsets
0  0.021692  (3 STRIPPY RICE FELTCRAFT)
1  0.039175  (6 RIBBONS RUSTIC CIGARS)
2  0.025146  (60 CAKE CASES VINTAGE CHRISTMAS)
3  0.035452  (60 TEATIME FAIRY CAKE CASES)
4  0.027034  (72 SWEETHEART FAIRY CAKE CASES)

4 rules = association_rules(
    frequent_items,
    metric='lift',
    min_threshold=1
)

print(
    rules[['antecedents',
        'consequents',
        'support',
        'confidence',
        'lift']]
)

antecedents \
0  (ALARM CLOCK BAKELIKE RED )
1  (ALARM CLOCK BAKELIKE GREEN)
```

```
MLProj.ipynb - Colab
colab.research.google.com/drive/1Gw_Lh5VdhfQB8q4yNO6xATSMCPnAz9#scrollTo=Y97oSFfIfpW
Ask Gemini
File Edit View Insert Runtime Tools Help
Commands + Code + Text Run all
Connect

1 frequent_items = apriori(
    basket,
    min_support=0.02,
    use_colnames=True
)

print(frequent_items.head())

support  itemsets
0  0.021692  (3 STRIPPY RICE FELTCRAFT)
1  0.039175  (6 RIBBONS RUSTIC CIGARS)
2  0.025146  (60 CAKE CASES VINTAGE CHRISTMAS)
3  0.035452  (60 TEATIME FAIRY CAKE CASES)
4  0.027034  (72 SWEETHEART FAIRY CAKE CASES)

2 rules = association_rules(
    frequent_items,
    metric='lift',
    min_threshold=1
)

print(
    rules[['antecedents',
        'consequents',
        'support',
        'confidence',
        'lift']]
)

antecedents \
0  (ALARM CLOCK BAKELIKE RED )
1  (ALARM CLOCK BAKELIKE GREEN)
```

```

print(
    rules[
        'antecedents',
        'consequents',
        'support',
        'confidence',
        'lift'
    ]
)

...
antecedents \
0 (ALARM CLOCK BAKELIKE RED )
1 (ALARM CLOCK BAKELIKE GREEN)
2 (ALARM CLOCK BAKELIKE RED )
3 (ALARM CLOCK BAKELIKE PINK)
4 (DOLLY GIRL LUNCH BOX)
...
71 (PINK REGENCY TEACUP AND SAUCER, GREEN REGENCY...
72 (ROSES REGENCY TEACUP AND SAUCER , GREEN REGEN...
73 (PINK REGENCY TEACUP AND SAUCER)
74 (ROSES REGENCY TEACUP AND SAUCER )
75 (GREEN REGENCY TEACUP AND SAUCER)

consequents support confidence \
0 (ALARM CLOCK BAKELIKE GREEN) 0.028599 0.604333
1 (ALARM CLOCK BAKELIKE RED ) 0.028599 0.671736
2 (ALARM CLOCK BAKELIKE PINK) 0.021368 0.451539
3 (ALARM CLOCK BAKELIKE RED ) 0.021368 0.646003
4 (SPACEBOY LUNCH BOX ) 0.022879 0.688312
...
71 (ROSES REGENCY TEACUP AND SAUCER ) 0.021045 0.847826
72 (PINK REGENCY TEACUP AND SAUCER) 0.021045 0.720887
73 (ROSES REGENCY TEACUP AND SAUCER , GREEN REGEN... 0.021045 0.701439
74 (PINK REGENCY TEACUP AND SAUCER, GREEN REGENCY... 0.021045 0.498084
75 (PINK REGENCY TEACUP AND SAUCER, ROSES REGENCY... 0.021045 0.564399

```

```

...
lift
0 14.194548
1 14.194548
2 13.650778
3 13.650778
4 18.119023
...
71 20.066300
72 24.027846
73 24.027846
74 20.066300
75 23.989564

[76 rows x 5 columns]

anomalies = rfm[
    rfm['DBSCAN_Cluster'] == -1
]

print("Number of anomalies:")
print(len(anomalies))

Number of anomalies:
45

```

10. Conclusion

This project successfully applied unsupervised learning techniques to perform customer segmentation and market basket intelligence. K-Means, Hierarchical Clustering, and DBSCAN effectively identified customer groups and anomalies. PCA improved visualization and interpretability. The generated customer personas and purchasing insights can assist businesses in making data-driven marketing decisions and enhancing customer engagement.